multiVector.net

A live, expression-driven
Geometric Algebra visualization environment

Étienne Harquin

May 28, 2026

PhD candidate

Why this matters

The GA tooling gap

Geometric Algebra is winning the theoretical battle in graphics, robotics and physics. But the tooling story has not kept pace.

- ganja.js — powerful but bare; every demo is hand-rolled.
- GAlgebra / Clifford.jl — symbolic, no live interaction.
- CGA viewer — single algebra, fixed UI.

No project today gives the user:

- a uniform expression language across algebras,
- an interactive canvas where every object is draggable, and
- a clean way to define their own primitives.

The opportunity

GA needs a **Desmos-style standard** for visualizing multivector constructions — but mathematically faithful, algebra-pluggable, and open.

multiVector.net aims to be that standard.

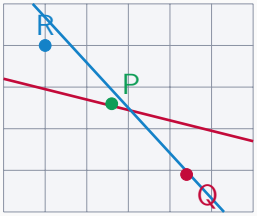
- One language, many algebras.
- Expressions *are* the construction — drag a point, every dependent object updates.
- User-extensible: define your own primitives in the language itself.

What it is

In one screenshot

A web application (React + Vite, SVG-rendered, `ganja.js` for GA primitives) where the entire construction lives in a left-hand expression panel.

- Type an expression ⇒ object appears.
- Drag the object ⇒ the expression updates.
- Edit the expression ⇒ the canvas redraws.
- Persistent: `localStorage` + named saved graphs.



Canvas (SVG, dark/light themed)

The expression language

Every line is parsed, classified, evaluated, and rendered.

```
P = point(1, 1)              # free draggable point
Q = point(2, 2)
L = P & Q                    # join: line through P and Q
M = exp(0.5 * e12)           # rotor (motor)
Q'= M >>> P                  # sandwich transform
t = 0.5                      # animatable scalar slider
T = exp(t * (e01 + e02))     # translator parameterised by t
```

Operators (tight → loose): ! ~ ≻ ^ & | § ≻ * / ≻ >>> ≻ + −

Type is a property of value, not syntax

Any expression is legal — its kind is determined by classifying the computed multivector:

kind	rendered as
finite point	dot
ideal point	ring on horizon ellipse
line	infinite line
ideal line	dashed horizon ellipse
vector	arrow
bivector	oriented loop, area $\propto \sqrt{\lvert b \rvert}$
rotor	arc + angle label
list	polygon outline (if all points)

Stroke widths and radii scale with the multivector's weight, so `5*e12` renders *five*

The architectural contribution

Pluggable algebra adapters

Each algebra is a single file exposing one spec object:

spec field	role
`Algebra`, `arraySize`	ganja.js binding
`bladeIndex`, `parseBladeName`	basis & parser hook
`classifyMV`	value → kind
`normalizeMVFinit/Ideal`	finite & ideal norms
`getRenderPlan`	value → SVG primitive
`supportedNodeTypes`	gates parser branches
`INITIAL_ITEMS`	showcase on load

The parser, evaluator, expression engine and node-type registry are factory functions bound to a spec — **adding a new GA is one new file**.

Already shipping: PGA(2,0,1), VGA(2,0,0), R(0,1,0)

Ganja delegation, single sign convention

Every GA primitive routes through ganja's built-in:

operation	routed to
dual `!A`	`Algebra.Dual`
reverse `~A`	`Algebra.Reverse`
sandwich `M >> A`	`Algebra.sw`
exponential `exp(V)`	`V.Exp()` (typed copy)
`sqrt(motor)`	log → halve → exp
finite norm	`Algebra.Length`
ideal norm	`Algebra.Length(Dual)`

No parallel implementations to drift. Adding a new algebra inherits all of GA semantics for free — provided ganja supports it.

Features (today)

Lists as first-class values

```
L = [P, Q, M >>> P, M*M >>> P]    # any types mix
~L                                # broadcast over list
M >>> L                           # transform every element
L1 ^ L2                           # elementwise wedge
len(L), L[i], L[i:j]              # length, index, slice
```

All unary / binary operators broadcast or work elementwise. All-point lists draw a dashed polygon.

Smart norms, postfix accessors

```
|P & Q|                  # smart norm (auto finite/ideal)
A.norm    A.inorm       # explicit finite / ideal norm
abs(s)                  # scalar absolute value
A.e12                   # blade coefficient extraction
A.e21                   # permuted blades: A.e21 = -A.e12
```

Plus norm / inorm toggle buttons per row that normalise the underlying value and propagate to dependents.

User-defined functions (just landed)

```
dist(A, B) = |A & B|          # function definition
midpoint(A, B) = (A + B) / 2
ln(A, B)   = A & B            # parameters are any object

d  = dist(P, Q)               # call anywhere
m  = midpoint(P, Q)
L  = ln(P, Q)
d2 = 2 * dist(P, Q) + 1       # nested in arithmetic
```

Functions are values — captured globals propagate reactively, recursion is allowed with a depth cap, builtin-name collisions are rejected at parse time.

Animation + interactivity

Scalars are sliders

- interval (`min, max, step`) per scalar,
- play ▷ / pause; modes: pingpong, repeat, once, infinite,
- speed levels 0.25× to 8×,
- any dependent object animates in lock-step.

Direct manipulation

- drag points, vector tail / tip independently,
- lock per item, hide per item,
- undo / redo (Ctrl-Z / Y), 100-step history with high-frequency drag coalescing,
- appearance per item: color, opacity, size, shape, label.

Polish that matters for adoption

- **Light / dark theme** — every SVG color goes through CSS variables; theme switch cross-fades.
- **Adaptive grid** — major + minor (5-subdivision) gridlines, axis labels reposition when off-screen.
- **Persistence** — autosave to `localStorage` keyed per algebra; `Save` / `Open` named graphs.
- **Share** — encode the whole graph in a URL fragment for a one-click link.
- **Help modal** — full expression reference inline, no docs round-trip needed.
- **Error UX** — duplicate labels, invalid syntax, builtin collisions all surface as a red pastille + a single-line message; the row's value display is suppressed cleanly.

Why this matters for the PhD

A platform, not a demo

Most GA visualisation work is one-off: a fixed algebra, a fixed scene, a paper figure.

multiVector.net is general infrastructure.

- Teaching: live derivations in a lecture, students fork the URL.
- Research: prototype a construction in minutes, save the JSON alongside the paper, drop a URL into reviewer correspondence.
- Community: a shared substrate where each algebra adapter and each saved graph compounds.

Concrete academic contributions

1. **The algebra-adapter pattern.** A reproducible recipe for hosting any Clifford algebra in a single interactive UI.
2. **The expression language.** A small but complete surface — binary GA operators, lists, smart norms, postfix accessors, label templates, user functions.
3. **A reference implementation.** Open, web-native, no install — every figure in a future paper can be a live URL.
4. **A library of saved constructions.** The JSON format is stable; each saved graph is a citable artefact.

Each of these is a paper-sized contribution. Together they are the beginning of a community standard.

Roadmap

- More algebras: CGA, STA, 3D PGA — each is one adapter file.
- Per-graph published URL → versioned share links.
- Inline LaTeX rendering of expressions and labels.
- Replay of construction history as an animation (already have the undo / redo log — just expose it).
- Export: SVG, animated GIF, PDF figure with attached `.json`.
- A gallery: curated saved graphs as a discoverable corpus.

The ask

I would like to position the publication of **multiVector.net** — the system, the language, and the adapter pattern — as a contribution of my thesis.

Short-term targets I can argue for:

- a tool paper (e.g. AGACSE, CGI tools track, or JOSS),
- a tutorial paper using the platform for derivations,
- a community-standard proposal (saved-graph JSON format).

Live demo: `http://localhost:5174`

Thank you, Vincent.

Questions, sharpened criticism, and re-direction welcome.